

§ 6.5 图遍历算法概述

图算法是个庞大的家族，其中大部分成员的主体框架，都可归结于图的遍历。与5.4节中树的遍历类似，图的遍历也需要访问所有顶点一次且仅一次；此外，图遍历同时还需要访问所有的边一次且仅一次——尽管对树而言这显而易见——并对边做分类，以便后续的处理。

实际上，无论采用何种策略和算法，图的遍历都可理解为，将非线性结构转化为半线性结构的过程。经遍历而确定的边类型中，最重要的一类即所谓的树边，它们与所有顶点共同构成了原图的一棵支撑树（森林），称作遍历树（**traversal tree**）。以遍历树为背景，其余各种类型的边，也能提供关于原图的重要信息，比如其中所含的环路等。

图中顶点之间可能存在多条通路，故为避免对顶点的重复访问，在遍历的过程中，通常还要动态地设置各顶点不同的状态，并随着遍历的进程不断地转换状态，直至最后的“访问完毕”。图的遍历更加强调对处于特定状态顶点的甄别与查找，故也称作图搜索（**graph search**）。

与树遍历一样，作为图算法基石的图搜索，本身也必须能够高效地实现。幸运的是，正如我们马上就会看到的，诸如深度优先、广度优先、最佳优先等基本而典型的图搜索，都可以在线性时间内完成。准确地，若顶点数和边数分别为 n 和 e ，则这些算法自身仅需 $O(n + e)$ 时间。既然图搜索需要访问所有的顶点和边，故这已经是我们所能期望的最优的结果。

§ 6.6 广度优先搜索

6.6.1 策略

各种图搜索之间的区别，体现为边分类结果的不同，以及所得遍历树（森林）的结构差异。其决定因素在于，搜索过程中的每一步迭代，将依照何种策略来选取下一接受访问的顶点。

通常，都是选取某个已访问到的顶点的邻居。同一顶点所有邻居之间的优先级，在多数遍历中不必讲究。因此，实质的差异应体现在，当有多个顶点已被访问到，应该优先从谁的邻居中选取下一顶点。比如，广度优先搜索（**breadth-first search, BFS**）采用的策略，可概括为：

越早被访问到的顶点，其邻居越优先被选用

于是，始自图中顶点 s 的BFS搜索，将首先访问顶点 s ；再依次访问 s 所有尚未访问到的邻居；再按后者被访问的先后次序，逐个访问它们的邻居；...；如此不断。在所有已访问到的顶点中，仍有邻居尚未访问者，构成所谓的波峰集（**frontier**）。于是，BFS搜索过程也可等效地理解为：

反复从波峰集中找到最早被访问到顶点 v ，若其邻居均已访问到，则将其逐出波峰集；否则，随意选出一个尚未访问到的邻居，并将其加入到波峰集中

不难发现，若将上述BFS策略应用于树结构，则效果等同于层次遍历（5.4.5节）——波峰集内顶点的深度始终相差不超过一，且波峰集总是优先在更浅的层次沿广度方向拓展。实际上，树层次遍历的这些特性，在一定程度上也适用于图的BFS搜索（习题[6-7]）。

由于每一步迭代都有一个顶点被访问，故至多迭代 $O(n)$ 步。另一方面，因为不会遗漏每个刚被访问顶点的任何邻居，故对于无向图必能覆盖 s 所属的连通分量（**connected component**），对于有向图必能覆盖以 s 为起点的可达分量（**reachable component**）。倘若还有来自其它连通分量或可达分量的顶点，则不妨从该顶点出发，重复上述过程。